Katholieke
Universiteit
Leuven

**Department of
Computer
Science**

PRIVACY IMPACT ASSESSMENT OF THE DOCSPELL

H00Y3a Privacy and Big Data

Peter Keseru (r0928168)
Yu Tao (r0921203)
Tao Liu (r0868419)
Jiaxi Zheng (r0918315)

Academic year 2025–2026

Table of Contents

1. Introduction **3**

2. Description of the application **3**

2.1 Information flow 4

2.2 Functionality of the application 5

2.3 System architecture 6

2.4 Data collection and handling 7

2.5 Stakeholders 8

3. LINDDUN Privacy Threat Analysis **9**

3.1 Data Flow Diagram (DFD) 9

3.2 Possible Threats Elicitation 9

4. Identified Privacy Threats **13**

4.1 Unintended Sensitive Information Metadata Extraction 13

4.2 Indexing of Full Document Text 13

5. Testing **14**

5.1 Verification of NLP Filtering 14

5.2 Incomplete Deletion 15

5.3 Verification of Document Upload and Job Queue Privacy 16

6. Recommendations **17**

6.1 Reducing Exposure Through Metadata and Classification Controls 17

6.2 Securing the Full-Text Search Index 17

6.3 Reducing Privacy Exposure in Email Import and Processing Queues 18

6.4 Clarifying Trust Boundaries in Local and Distributed Deployments 18

7. Conclusion **19**

Disclaimer **20**

References **21**

Appendix **22**

1. Introduction

In the era of digitalization, Personal Document Management Systems (PDMS) are widely used by individuals and organizations to archive and manage digital documents such as invoices, contracts, medical records, and personal correspondence. Docspell is a prominent open-source solution in this domain, offering features such as full-text search, automatic tagging, and optical character recognition (OCR) (Kettner et al., 2025). However, the very nature of a PDMS requires it to ingest, process, and store highly sensitive Personally Identifiable Information (PII). Due to the aggregation of financial, legal, and health-related data within a single platform, a PDMS represents a high-value target for privacy and security breaches. As a result, privacy risks and regulatory considerations, including compliance with frameworks such as the General Data Protection Regulation (GDPR), are a critical aspect of the system's design and deployment. This report therefore examines the privacy implications of Docspell through a structured Privacy Impact Assessment (PIA).

2. Description of the application

Docspell is an application managing documents coming from different sources (uploaded manually, by scanning, from email or through the command line). Apart from storing, it comes with different features, for instance, OCR, text analysis, the possibility to organise documents into different containers (eg. "folders", "persons", "organisation") ensuring an order and more security. In addition, the app may provide convenience, for example, by predicting the "tag" of an uploaded document through a trainable algorithm.

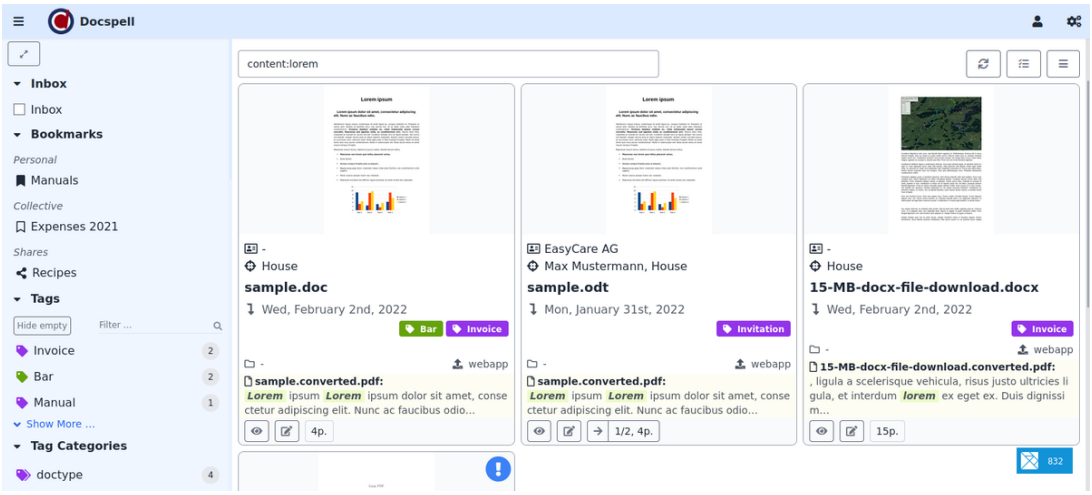


Figure 1 : Docspell Web dashboard.

2.1 Information flow

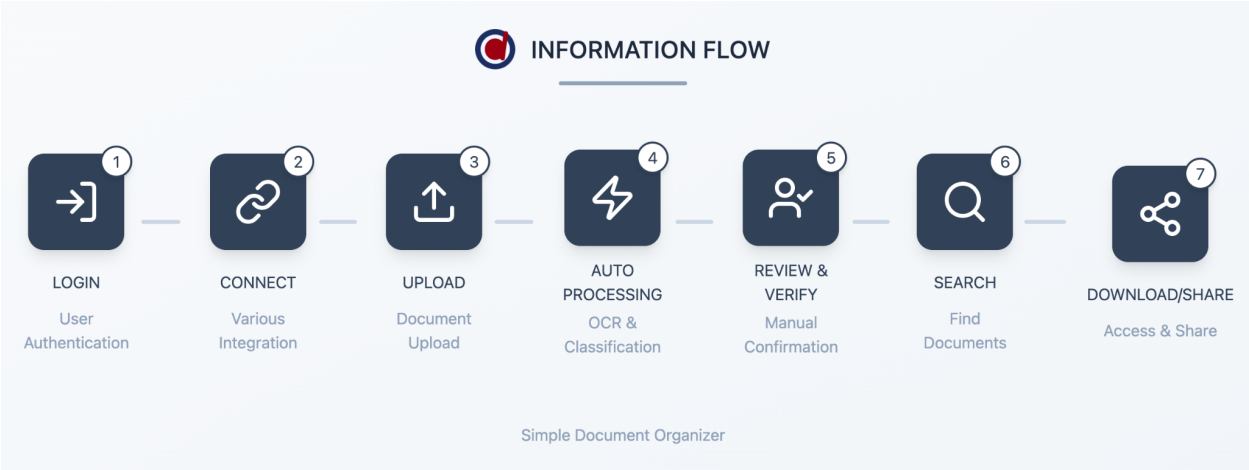


Figure 2: User information flow in Docspell

From a user perspective, interaction with Docspell follows a typical document lifecycle. Users begin by authenticating to the system, establishing a personalized session. Once logged in, they can connect one or more document sources, such as scanners, email inboxes, mobile devices, or command-line interfaces. Documents are then uploaded or automatically fetched into the system through these channels. After ingestion, documents enter an automated processing pipeline. During this phase, textual content is extracted using optical character recognition (OCR), and additional metadata is generated or inferred. The processed documents and associated metadata are stored centrally and indexed to enable search and retrieval. Users can subsequently review, modify, and refine metadata before accessing documents through filters or full-text search. Depending on access rights, documents may finally be downloaded or shared with other users. While this sequence reflects a common workflow, Docspell allows flexible interaction, enabling users to engage with the system in varying orders.

2.2 Functionality of the application

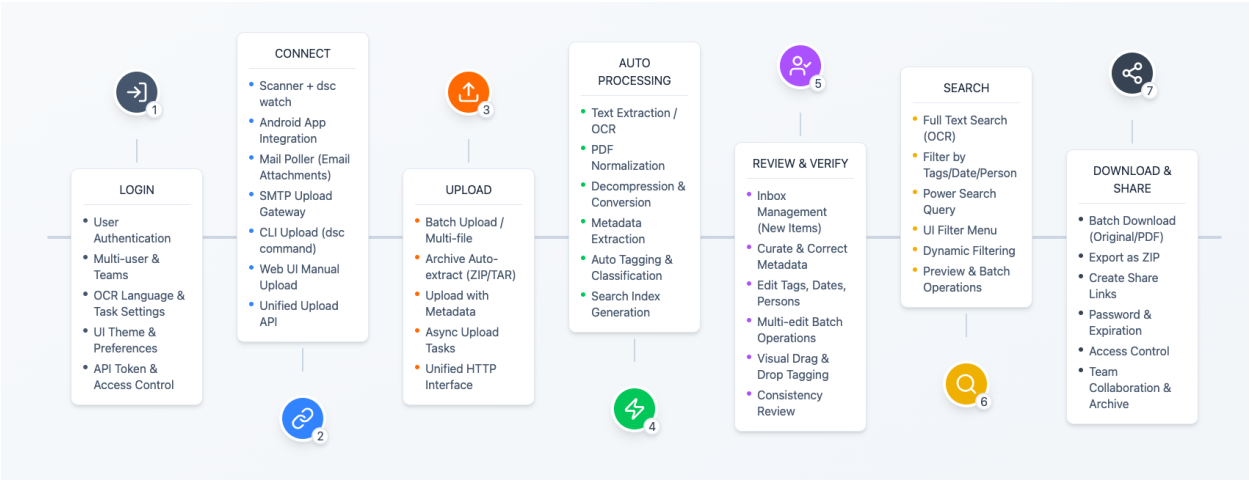


Figure 3: Functional overview of Docspell by information flow

Building on the information flow described above, Docspell's functionality is summarized in Figure 4 based on its official documentation (Docspell, n.d.) and structured around a user-oriented document lifecycle. The application combines essential features—secure user authentication, structured storage, metadata management, and full-text retrieval—with a strong emphasis on automation and workflow efficiency. Through multi-source ingestion and semantic processing, Docspell transforms document management from a passive archive into an active processing environment.

One of its key strengths is its connectivity flexibility. Users are not restricted to a single upload method, but can integrate the system into existing workflows. For example, scanners can be linked via a watch utility that automatically detects and archives new files. Mobile users can scan documents with Android applications and share them directly with the Docspell client. For email-based workflows, Docspell can fetch documents from designated mailbox folders or accept uploads via email attachments through a configured SMTP endpoint. Command-line support further enables automated and script-based interactions. Beyond ingestion, Docspell supports intelligent metadata inference. Uploaded files are normalized, converted into searchable PDFs, and processed to extract text. The system detects entities such as names and organizations and links them to existing records, attaching relevant tags or custom fields. This hybrid approach—combining automated inference with user refinement—allows metadata to be defined during upload or adjusted later, improving consistency and discoverability over time. The search and retrieval layer of Docspell converts the archive into an interactive knowledge space. In addition to basic searches, the system supports full-text and fuzzy matching, boolean queries, and structured filters. Users can save complex queries as bookmarks, enabling precise retrieval scenarios, such as identifying contracts that reference a specific organization within defined constraints. Through visual tag filtering and semantic queries, Docspell enables users to explore and understand their document collections rather than merely browse stored files.

2.3 System architecture

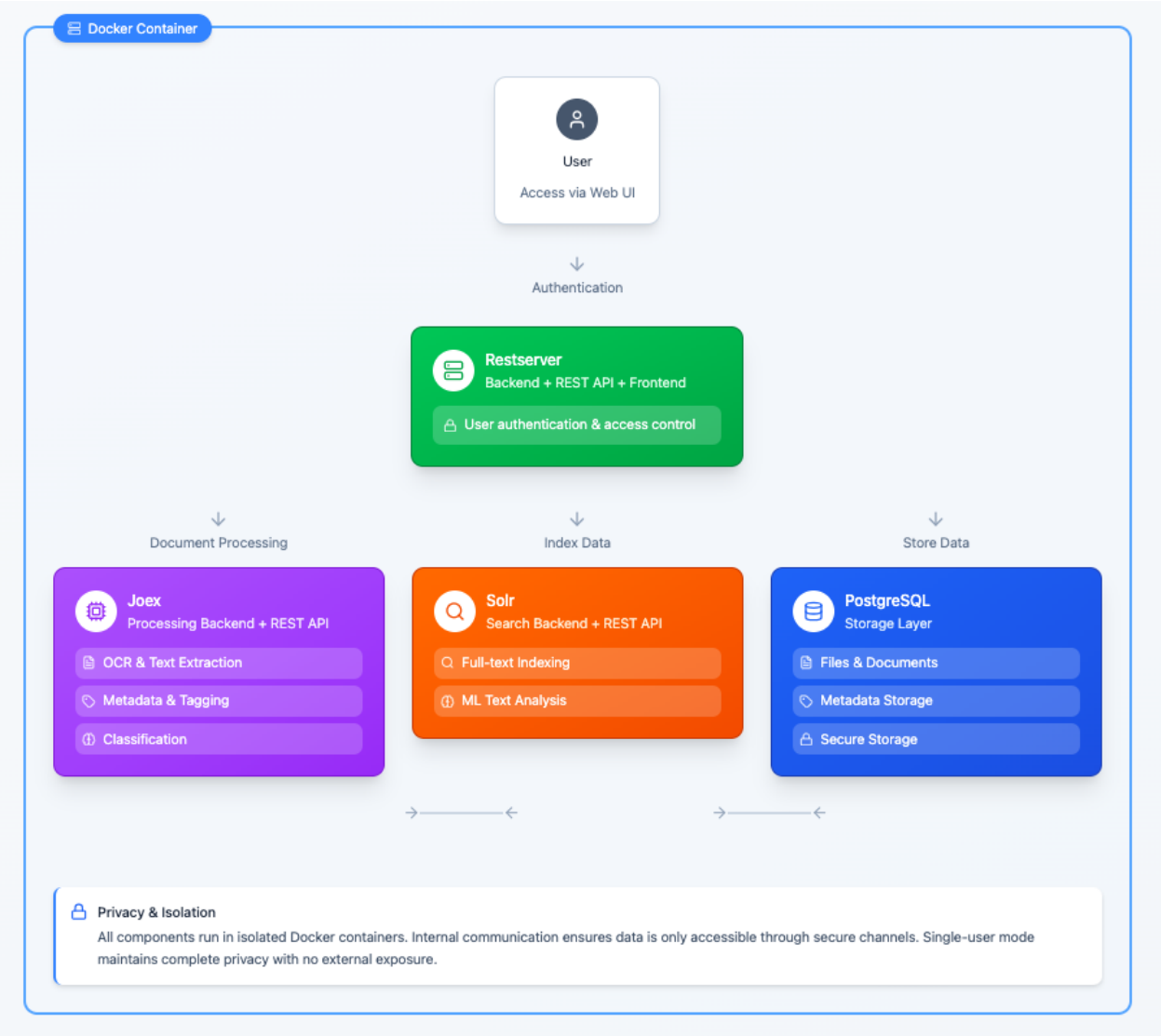


Figure 4: Docspell system architecture and components

Docspell is deployed in Docker, and has multiple components, which are Restserver, Joex, Solr, and PostgreSQL. The Restserver (back end + Rest API + frontend) handles user authentication, ensuring that only users with the correct credentials can access uploaded files. Joex (a server with its own Rest API) manages uploaded documents by performing OCR, extracting metadata, tagging, and classifying files. Solr is a NoSQL search engine, which can index data internally for search and performs text analysis through ML (PostgreSQL as a search engine is a supported alternative). All metadata and by default all uploaded files are stored in a database (by default PostgreSQL). Since all components run inside Docker containers, they are isolated from the host system. This architecture maintains privacy by ensuring that information is only accessible through the internal communication among Restserver, Joex, the search engine, and the database (and optionally a file system).

2.4 Data collection and handling

Docspell collects user-uploaded documents (e.g., PDFs, images, scans), which form the primary data processed by the system. From these documents, textual content is extracted using OCR and text extraction mechanisms and is used to generate a search index. Based on the extracted text, Docspell derives structured metadata, including document titles, tags, correspondents, and dates, which are used for classification, organization, and filtering. In addition, Docspell stores user account and permission data to enforce authentication, authorization, and access control in single-user and multi-user deployments. Together, these data are processed solely to enable document storage, retrieval, automated organization, and correct system functionality.

Docspell does not operate a central service and does not collect data centrally. All data processing occurs within the deployment environment. The system operator (self-hosted user or third-party hosting provider) is responsible for security configuration, encryption, backups, and legal compliance, including GDPR obligations. When Docspell is hosted by an external provider, that provider's privacy policy and compliance guarantees apply at the infrastructure level.

To support data organization and efficient retrieval, Docspell employs a two-layer data handling model that separates original documents from derived metadata. Uploaded files are preserved in their original form and stored either directly in the database or, if configured, in external storage such as a filesystem or S3-compatible storage. Extracted text and structured metadata are stored separately in the relational database to enable indexing and querying. Document processing tasks, including OCR, text extraction, and classification, are executed atomically to ensure consistency and to prevent partial or inconsistent updates. Each document and its associated metadata are linked to specific user and organization identifiers, thereby enforcing access control and preventing unauthorized access.

2.5 Stakeholders

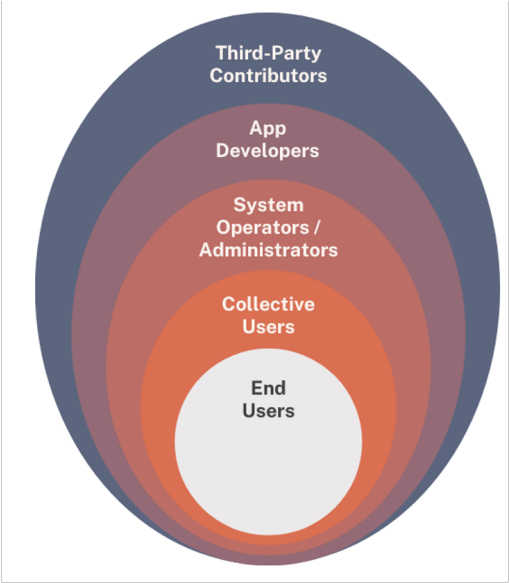


Figure 5: Docspell functionalities by flow

End-Users: The primary stakeholders are the end users who upload, manage, and search documents. They rely on Docspell to securely store sensitive files and metadata. Users expect strong access controls and accurate processing while keeping their data private.

Collective users: Collective users are stakeholders in scenarios where multiple individuals share the same document repository. Each user has a personal login. All users in a collective have equal access to the shared files. Multiple users in the same LAN can access Docspell using the administrator's local IP address. If they want to access it via the internet, additional configuration, such as setting up a VPN, is needed. The people who deploy and manage Docspell are responsible for data safety, especially when allowing multiple users and exposing the system over a network. They need to properly configure Docker, networking, storage, and backups to prevent potential leaks.

System Operators / Administrators: This stakeholder is the individual or entity that physically deploys, configures, and maintains the Docspell instance (e.g., setting up Docker, managing storage, ensuring backups, configuring network access). They are the ultimate party responsible for data safety, regulatory compliance (like GDPR), and overall system availability for the end-users they serve.

App developers: The Docspell development team is responsible for maintaining the system's privacy and security features. They ensure that the architecture enforces proper isolation and prevents unauthorized access.

Third-party contributors: Finally, plugin or integration developers are stakeholders who extend Docspell's functionality. While contributing, they must ensure that any added features do not compromise privacy or security, especially for features that interact with the internet.

3. LINDDUN Privacy Threat Analysis

3.1 Data Flow Diagram (DFD)

This Data Flow Diagram (DFD) illustrates the key processes, data stores, and external entities involved in document processing, from user upload to processing and storage. The architecture relies on a separation of concerns between the web-facing Restserver, the asynchronous processing component Joex, and the storage layers (PostgreSQL for metadata and Solr for full-text search).

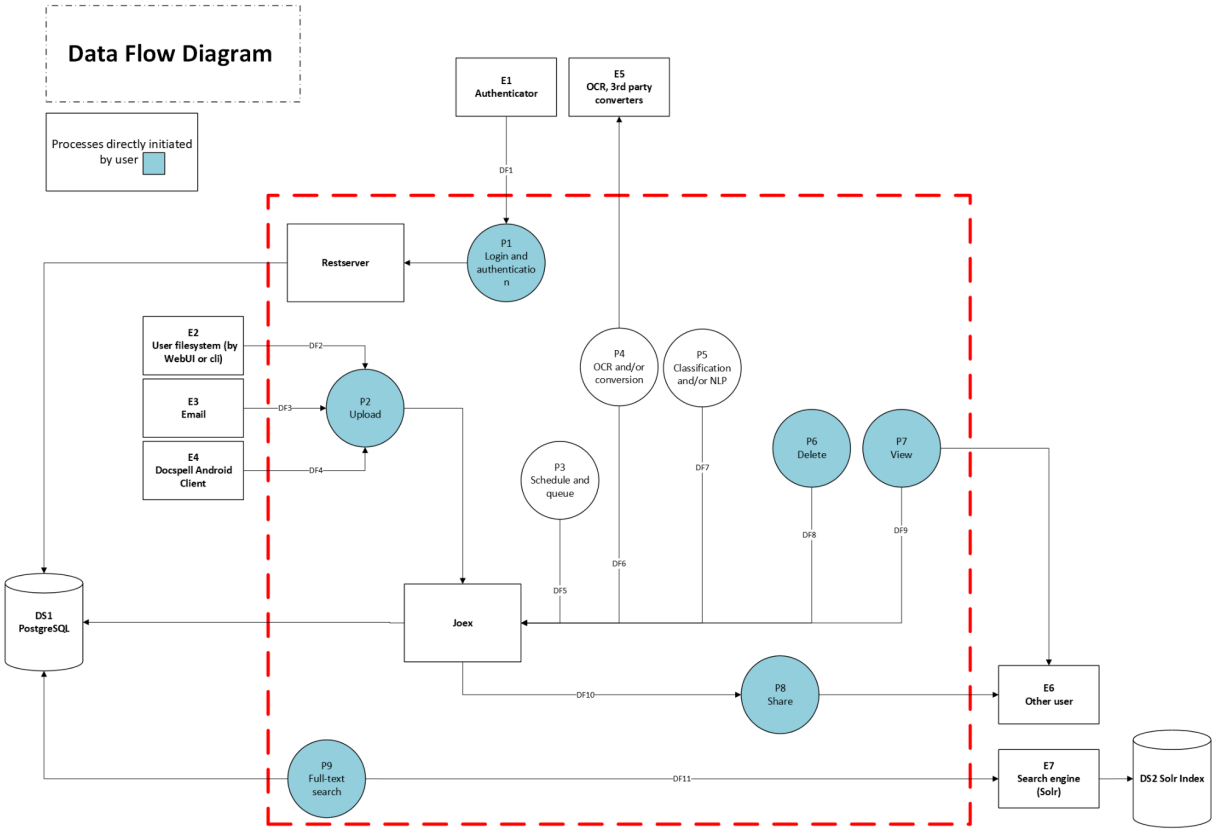


Figure 6: Docspell Data Flow Diagram.

3.2 Possible Threats Elicitation

The following table details possible privacy threats identified by applying the LINDDUN framework to the DFD in Figure 6. To ensure completeness and correct categorization, all identified threats were validated against the official LINDDUN threat trees. Each threat was mapped to its corresponding branch in the tree.

Code	Name	Description	Mapped LINDDUN Tree Branch	Component
L.1	Linking user identity via third-party OCR	If API requests include identifiers (user ID, username), the external OCR provider can create a user profile by linking multiple uploaded documents to the same user.	L.1.1 Unique Identifier + L.2.1 Through Combination	Jobs → E5
L.2	Document-to-user link through search index	The Solr index (DS2) connects link text to document IDs, which in turn link to user information. A breach of DS2 Solr text could allow an attacker to link multiple documents to a single user without revealing their identity.	L.1.1 Unique Identifier + L.2.1 Through Combination	P9 → DS2
L.3	Linking through uploaded and shared documents	Docspell maintains logs of uploads and sharing, which reveal who interacted with which documents and when. An attacker could infer interpersonal or organisational relationships from these patterns.	L.2.2.2 Profiling a group of individuals	System Logs → Jobs → DS1

Table 1: Linkability Threats

Code	Name	Description	Mapped LINDDUN Tree Branch	Component
I.1	Identifiable content exposed to third-party OCR	Uploaded documents often contain personal identifiers, and sending them to an external OCR provider exposes these details, increasing identifiability risks.	I.1.1 Processing of Identified Data	Jobs → E5 (External OCR)
I.2	Identifiable file names reveal user identity	File names can contain personal information (e.g., “Alice-Payroll.pdf”, “Bob-MedicalReport.jpg”), allowing systems to identify the document owner without opening the file.	I.1.2 Identified Information in Metadata	E2/E3/E4 → Jobs

I.3	Identifiable email metadata	(from, to, cc) may contain personal identifiers that make documents directly to specific individuals.	I.1.2 Identifiers in metadata	E3 → Jobs

Table 2: Identifiability Threat

Code	Name	Description	Mapped LINDDUN Tree Branch	Component
NR.1	Upload actions are attributable	The system records which user uploaded each document, creating permanent evidence of user actions.	Nr.1 Attributable Data Evidence → Nr.1.1 Data	E2/E3/ E4 → Jobs
NR.2	Sharing actions leave traceable logs	Sharing a document stores the identity of the user who initiated the share, preventing plausible deniability.	Nr.1 Attributable Data Evidence → Nr.1.1 Data	P8 → System logs
NR.3	Processing tasks are user-linked	OCR and classification jobs record which user triggered processing, making activities non-repudiable.	Nr.1 Attributable Data Evidence → Nr.1.1 Data	Jobs → P2/P3/ P4/P5

Table 3: Non-repudiation Threat

Code	Name	Description	Mapped LINDDUN Tree Branch	Component
DT.1	Detectability of document upload patterns	An external observer can infer when a user uploads a document by analyzing traffic bursts or timing patterns, revealing sensitive behavioral information without accessing the content.	D.1 Observed Communications	E1/E2/ E3 → Jobs
DT.2	Detectability through search index access patterns	Attackers with access to logs, metrics, or timing information may identify user interests or workflows based on document access patterns.	D.2 Application side effect	P9 → DS2

DT.3	Detectability through job queue activity	A passive observer of the job queue can detect user activity, with peaks indicating routines or organizational behavior patterns.	D.2 Application side effect	Jobs → P2/P3/ P4/P5

Table 4: Detectability Threat

Code	Name	Description	Mapped LINDDUN Tree Branch	Compo nent
D.1	External OCR receives full document	Full document content is sent to an external OCR service, exposing sensitive data outside the trusted environment.	DD.4.1.1 Predetermined Set of Parties	Jobs → E5
D.2	Sensitive data included in metadata	NLP/Classifiers may extract sensitive info and store it as metadata, increasing exposure.	DD.3.1 Treatment / Analysis / Enrichment / Transformation	Jobs → P5 → DS1
D.3	Search index contains all document text	Solr index stores unencrypted full text, allowing complete data exfiltration if compromised.	DD.1.2 Data Type Granularity	P9 → DS2

Table 5: Disclosure of Information Threats

Code	Name	Description	Mapped LINDDUN Tree Branch	Compon ent
U.1	User unaware data goes to external OCR	Users uploading files do not know documents are being processed by a third-party service.	U.1.1 Unawareness as Data Subject	E1/E2/E 3 → Jobs → E5

Table 6: Unawareness Threats

Code	Name	Description	Mapped LINDDUN Tree Branch	Component
N.1	OCR provider may violate GDPR	Outsourcing OCR to non-EU providers without a DPA can be illegal.	Nc.1.1.3.2 Lawfulness Problems	Jobs → E5

			Not Related to Consent	
N.2	Incomplete deletion violates GDPR	Deletion does not remove content from all storage components, violating the Right to be Forgotten.	Nc.1.1.4 Violation of Storage Limitation Principle	P6 → DS1/DS2

Table 7: Non-compliance Threats

4. Identified Privacy Threats

From the LINDDUN analysis and further exploration, the key privacy issues below have been identified and further discussed.

4.1 Unintended Sensitive Information Metadata Extraction

Docspell automatically generates metadata from the text extracted from uploaded documents, which includes notes, extracted text segments, inferred attributes, and system-generated labels. While these labels are attached to the document, they are distinct from user-created tags and do not contribute to the training of Docspell's classification algorithms, which depend on user-defined tags. Before confirming an upload, users have the option to remove the extracted text used to create some of this metadata, but they cannot remove the suggested auto-generated labels at that stage. The documentation lacks clarity on whether these labels are stored in the main database, maintained solely in the search index (such as Solr), or how they are exposed to other users. However, it is likely that any user who can access the document can also see the associated suggestions. While these automatically created elements serve to enhance search capabilities, automation, and document organization, they also pose privacy risks by potentially surfacing sensitive information that existed only in the document body into structured metadata fields or suggested labels, thereby increasing its visibility and searchability. The fact that certain auto-generated metadata cannot be edited or deleted prior to confirmation means that users may inadvertently expose information that they cannot fully control. This situation raises important questions regarding the persistence, propagation, and visibility of such metadata, as well as the users' ability to understand and manage these effects. Although Docspell provides UI indicators to show which metadata fields can be altered, it does not clearly communicate the practical immutability of the auto-generated labels. This lack of transparency, combined with the automatic transition of extracted text into structured fields, heightens the risk of unintended disclosure and limits users' control over sensitive or contextual information.

4.2 Indexing of Full Document Text

The full-text search engine (Solr) functions as one of the most critical and, simultaneously, one of the most privacy-sensitive components of the entire Docspell system. Solr stores complete, unencrypted text extracted from all uploaded documents, along with enriched metadata, classification results, access patterns, and document–user relationships. This concentration of highly sensitive information transforms Solr into a single point of catastrophic failure: any compromise of the index would immediately reveal the contents of every document stored in the system. In addition to exposing raw information, such a breach would also uncover linkability patterns, allowing attackers to correlate documents belonging to the same individual, as well as detectability signals in the form of query logs and access traces. The index therefore represents a disproportionate privacy risk, far exceeding that of other components, because it aggregates not only data but also the behavioural and relational insights derived from that data. Consequently, the index poses a disproportionate privacy risk compared to other components, as it aggregates not only data but also insights related to user behavior and relationships. Given its potential impact, Solr requires special scrutiny within the architecture. Moreover, it is not the only storage layer lacking built-in protection; Docspell's PostgreSQL database, which stores core metadata, user accounts, and document associations, is also unencrypted by default and does not support full encryption through native application mechanisms.

5. Testing

To validate the identified privacy threats and proposed mitigations, the project uses the pytest framework to create a suite of black-box integration tests. These tests interact with the Docspell instance exclusively through its public REST API, treating the system as a black box and asserting behaviour from a user's perspective. All tests follow the standard Arrange–Act–Assert (AAA) structure.

5.1 Verification of NLP Filtering

This test case highlights the risk of Docspell's NLP pipeline extracting sensitive personal identifiers and exposing them as globally visible metadata, such as tags, which could compromise privacy by making sensitive information (e.g., IBAN-like or SSN-like strings) accessible in the user interface. To verify that the default NLP configuration mitigates this risk, an automated regression test is conducted. The test begins by recording the existing set of global tags in the system. It then generates and uploads a document containing fabricated sensitive patterns, such as a fake Social Security Number. A synthetic document such as *sensitive_bank_statement.pdf* is created containing a unique, fabricated sensitive string ("987-654-321"). The test uploads this document to Docspell via the /api/v1/items endpoint and allows sufficient time for asynchronous processing—including OCR and the P2/P3 classification steps—to complete. The upload is done via the REST API, allowing time for Docspell's asynchronous OCR and NLP processing to complete. Once processing is finished, the test retrieves the updated list of tags. The expected outcome is that none of the sensitive patterns from the document appear in the resulting metadata. If the tag list remains unchanged—indicating no sensitive strings have been introduced—it confirms that the system filters such patterns effectively during NLP classification. Conversely, if any sensitive strings appear in the

metadata, the threat would be validated. Under the default Docspell configuration, the test passed, confirming that the NLP pipeline correctly ignored in Figure 7,8 the fabricated sensitive patterns and did not promote personal data into the global tag namespace. Continuous regression testing is recommended to ensure that future configuration or model changes do not compromise this protection.

```
============================================================================ test session starts ============================================================================
platform darwin -- Python 3.13.2, pytest-9.0.0, pluggy-1.6.0
rootdir: /Users/chen/Documents/docspell
collected 1 item

test_privacy_threats.py
--- (Fixture Setup) ---
  > POST http://localhost:7880/api/v1/open/auth/login
Login successful.
--- (Fixture Setup Complete) ---
(Setup) Created sensitive doc: test_doc_EP09FU3B1GSEL8Y.txt

--- RUNNING TEST: Threat #5 (Sensitive Data Leakage) ---
(Arrange) Test file: test_doc_EP09FU3B1GSEL8Y.txt
(Arrange) Sensitive strings to check: ['testtagep09fu3b1gsel8y', '987-654-321', '4111111111111111']
(Arrange) Getting list of tags BEFORE upload...
  > GET http://localhost:7880/api/v1/sec/tag
(Arrange) Found 0 tags: set()

(Act) Uploading test_doc_EP09FU3B1GSEL8Y.txt...
  > POST http://localhost:7880/api/v1/sec/upload/item (File: test_doc_EP09FU3B1GSEL8Y.txt, Type: text/plain)
(Act) Upload successful. Server response: {'success': True, 'message': 'Files submitted.'}
(Act) Waiting 15s for server to process and run NLP...
(Act) Getting list of tags AFTER upload...
  > GET http://localhost:7880/api/v1/sec/tag
(Act) Found 0 tags: set()

(Assert) Checking for new tags...
(Assert) New tags created by upload: set()

(Assert) Test PASSED: No sensitive data was leaked to the global tags.
--- TEST COMPLETE: Threat #5 ---
.

============================================================================ 1 passed in 15.27s ============================================================================
```

```
--- RUNNING TEST: Threat #5 (Sensitive Data Leakage) ---
(Arrange) Test file: threat5_case_049.txt
(Arrange) Sensitive strings to check: {'149591049', 'acc-0049', '149-59-1049', 'case049@example.com', '397317193973719
3', 'de02490100004900000004900'}
(Arrange) Getting list of tags BEFORE upload...
  > GET http://localhost:7880/api/v1/sec/tag
(Arrange) Found 0 tags: set()

(Act) Uploading threat5_case_049.txt...
  > POST http://localhost:7880/api/v1/sec/upload/item (File: threat5_case_049.txt, Type: text/plain)
(Act) Upload successful. Server response: {'success': True, 'message': 'Files submitted.'}
(Act) Waiting 2s for server to process and run NLP...
(Act) Getting list of tags AFTER upload...
  > GET http://localhost:7880/api/v1/sec/tag
(Act) Found 0 tags: set()

(Assert) Checking for new tags...
(Assert) New tags created by upload: set()

(Assert) Test PASSED: No sensitive data was leaked to the global tags.
--- TEST COMPLETE: Threat #5 ---
.

========================================== 50 passed in 110.15s (0:01:50) ==========================================
```

Figure 7,8: Terminal output showing the successful execution of the pytest suite. The ”1 passed” result confirms that no sensitive data was leaked into the global tags.

5.2 Incomplete Deletion

The second integration test validates Docspell's compliance with the GDPR “Right to be Forgotten” and specifically whether deletion operations (P5) propagate correctly to the search index (DS2). Complementing this search-based validation, an additional automated test verifies index removal at the Solr level. This test records the initial number of Solr indexes, uploads a simple text file, confirms the generated item and its metadata, and repeatedly polls Solr during upload, confirmation, and deletion. The expected pattern is an index count increase after ingestion and a full return to the initial count after deletion. A matching sequence Figure 9 demonstrates that the underlying Solr data structures are removed alongside the document,

confirming that deletion cascades correctly not only at the search-interface level but also within the storage layer that underpins DS2.

```
========================================= test session starts =========================================
platform win32 -- Python 3.11.0, pytest-9.0.1, pluggy-1.6.0
rootdir: C:\Users\keser\Desktop\kul\docspell-docker
collected 1 item

testing_lindunn.py
--- (Fixture Setup) ---
  > POST http://localhost:7880/api/v1/open/auth/login
Login successful.
--- (Fixture Setup Complete) ---

Initial count with no files uploaded: 19

  > POST http://localhost:7880/api/v1/sec/upload/item (File: lorem.txt, Type: text/plain)
Upload response JSON: {'success': True, 'message': 'Files submitted.'}

Count after upload

Uploaded item: {'groups': [{'name': '2025-11', 'items': [{'id': 'HHUWsLpnbXj-H4LPZKdvKjH-phnasgeVh8t-w3veG3csSnh', 'name': 'lorem.txt', 'state': 'created
', 'date': 1763734626436, 'dueDate': None, 'source': 'webapp', 'direction': 'incoming', 'corrOrg': None, 'corrPerson': None, 'concPerson': None, 'concEqu
ipment': None, 'folder': None, 'attachments': [], 'tags': [], 'customfields': [], 'relatedItems': [], 'notes': None, 'highlighting': []}]}], 'limit': 200
, 'offset': 0, 'limitCapped': False}

Item ID: HHUWsLpnbXj-H4LPZKdvKjH-phnasgeVh8t-w3veG3csSnh

Confirmation response JSON: {'success': True, 'message': 'Item data confirmed'}

Count after confirmation
Index updated, new count: 34

Delete response JSON: {'success': True, 'message': 'Item deleted'}

Count after deletion
Index updated, new count: 19
.

========================================= 1 passed in 33.48s =========================================
```

Figure 9: No Solr index is retained in the database after deletion of the file.

5.3 Verification of Document Upload and Job Queue Privacy

This test verifies that Docspell preserves privacy during document upload and job-queue processing, ensuring that user-identifying information is not exposed in either document metadata or internal processing queues. Two test files are uploaded through the REST API under an authenticated session, and their assigned document IDs are retrieved once ingestion has completed. In parallel, the job queue is queried to capture all active and pending jobs. The verification checks that no uploader information is present in the metadata of either document and that the document IDs are distinct from the job identifiers recorded in the queue. As shown in Figure 10, the uploaded documents receive unique identifiers, none of which overlap with job IDs, and the job queue contains only file names without any user identity or sensitive metadata. This confirms that Docspell cleanly separates job tracking from user data and maintains privacy during upload and background processing.

```
============================= test session starts ==============================
collecting ... collected 1 item

test.py::test_id PASSED                                                  [100%]=== Uploading Documents ===
Document 'doc1.txt' -> ID: VYwUbWcixRo-NLgw5iNdKyt-j74RPCjrZAt-HeHbgN6DiV, Uploader: None
Document 'doc2.txt' -> ID: Dz2NPsBHYd1-Uo8BiPxMUVS-4WvhGsztao7-d6SqTYxnjsB, Uploader: None

=== Checking Job Queue ===
State: progress — 1 jobs
  Job ID: 2cQku61xwg7-N4WKhqvznLS-3ajaez7FJP1-hjK6siCjkE2, Item ID: None, Item name: doc1.txt
State: completed — 0 jobs
State: queued — 1 jobs
  Job ID: 9sSxqXabZGX-wv4YrxJV8vK-BqbtZ55584q-5hFUiWJNXDH, Item ID: None, Item name: doc2.txt


======================== 1 passed, 1 warning in 0.27s ========================

Process finished with exit code 0
```

Figure 10: Test result with Document IDs, job IDs, and job queue details

6. Recommendations

6.1 Reducing Exposure Through Metadata and Classification Controls

Although the current evaluation shows that Docspell's default NLP configuration did not elevate synthetic sensitive patterns into metadata during testing, the NLP pipeline may still pose a risk under different models or configurations. To maintain data-minimization guarantees, Docspell should introduce an intermediate redaction layer to detect and suppress sensitive patterns (e.g., SSN-like or financial identifiers) before metadata is written into the primary database.

This can be complemented with a configurable ignore-list, allowing deployers to suppress domain-specific patterns. More robustly, moving from block-listing to a tag-category allow-list would reduce accidental PII extraction by ensuring that only recognized, predefined tag classes are ever generated. Metadata visibility should additionally be governed through fine-grained permissions (e.g., separate "metadata-view" and "metadata-admin" roles), ensuring that only authorized users can access or edit derived metadata and preventing unnecessary exposure of NLP-generated information. To ensure long-term correctness, the privacy-focused integration tests described in Section 6 should be incorporated into CI/CD regression testing, enabling early detection of accidental metadata leakage in future model updates or configuration changes.

6.2 Securing the Full-Text Search Index

The Solr search index contains a full copy of every processed document, and a compromise of this component alone would allow an attacker to exfiltrate all textual content. This warrants significantly stronger protections. At a minimum, the search index should be encrypted at rest, and its access must be restricted through network segmentation, hardened credentials, and

strict access policies comparable to those used for the primary relational database. To reduce linkability risks, behavior-related metadata such as access timestamps should be minimized or excluded from indexing altogether. Sensitive metadata fields should additionally be subject to selective encryption or pseudonymization to avoid unnecessary exposure within the index. Solr query permissions must also mirror the database's authorization model to prevent the search interface from becoming a privilege-bypass path. The index should not be exposed to untrusted networks, and the system should implement routine integrity checks to ensure that the component is not inadvertently accessible.

6.3 Reducing Privacy Exposure in Email Import and Processing Queues

When users import emails into Docspell, messages are automatically converted into PDF files, including contextual metadata such as senders, recipients, and subject lines. While selecting the "only import email attachments" option prevents the email body from being archived, subject information remains visible during the processing stage. In multi-user collectives, this results in email subjects being observable by other users in the shared processing queue, even when subject-based filters are applied. Although other users cannot access email account credentials or IMAP/SMTP configurations, partial disclosure of email topics may still constitute an unintended privacy exposure.

Since this behavior is inherent to the current user interface design rather than a testing or implementation flaw, mitigation should focus on access control and user awareness. It is therefore recommended to restrict processing-queue visibility such that email subjects are only visible to the importing user. Additionally, the system should provide explicit warnings prior to enabling email import, clearly informing users that subject metadata may become visible within a shared collective. Users should be encouraged to prefer attachment-only imports or narrowly defined subject filters to minimize inadvertent disclosure of private or sensitive information.

6.4 Clarifying Trust Boundaries in Local and Distributed Deployments

Due to its typical self-hosted deployment, Docspell's privacy guarantees heavily depend on the trustworthiness of those administering or hosting the system. Administrators have inherent access to all files within the system and can perform privileged actions such as password resets and metadata edits, creating a reliance on their reliability and accountability. Thus, it is essential that deployments ensure administrators are recognized as data processors under applicable privacy regulations. Docspell's architecture supports distributed multi-node deployment, where components like the REST server, JOEX job executor, and the DSC upload watcher may run on different machines. Each of these components requires direct access to a shared database and binary storage backend, such as a filesystem or S3, which means that any entity hosting a component has the technical capability to access all stored documents. This broad access brings additional privacy risks.

To mitigate unnecessary data exposure in multi-node deployments, Docspell should: 1) Enforce least-privilege access controls to minimize the need for components to read full document

content. 2) Provide clear documentation of trust assumptions for deployers, explicitly outlining which roles or nodes have full data visibility. 3) Support encrypted transport and mutual authentication (e.g., TLS with client certificates) between distributed components. 4) Optionally introduce component-level access scopes to prevent non-trusted third parties from running arbitrary components against the shared database. By clarifying and constraining these trust boundaries, distributed deployments can remain privacy-preserving while ensuring administrative access aligns with GDPR accountability requirements.

For instance, while Docspell performs OCR locally using Tesseract (via TextExtractionSuite.scala) by default, some deployments may choose to integrate external OCR services. In such cases, full document contents may be transmitted to third-party providers, effectively shifting parts of the processing outside the original trust boundary. This change alters the allocation of responsibility, as external providers may act as additional data processors under GDPR. To ensure accountability in such extended deployments, any use of external OCR should be explicitly enabled by the deploying organisation and made transparent to end users. Users should be informed when their documents are processed externally and, where appropriate, given a meaningful opportunity to consent, either on a per-document basis or through a dedicated configuration option. Integrations with external OCR providers should minimise the transmission of user identifiers and system metadata, and organisations should ensure that appropriate contractual safeguards, such as a Data Processing Agreement and compliant international transfer mechanisms, are in place.

7. Conclusion

This Privacy Impact Assessment evaluated Docspell as a self-hosted Personal Document Management System, focusing on privacy risks arising from automated document processing, metadata extraction, and long-term storage. By analyzing the system's information flows and applying the LINDDUN framework, the assessment identified key risks related to unintended metadata exposure, full-text indexing, and the concentration of sensitive personal data within central system components.

The analysis shows that while Docspell benefits from local deployment and the absence of centralized data collection, its automated processing and search capabilities introduce non-trivial privacy risks. Automatically generated metadata and the full-text search index increase the potential impact of data breaches and may limit user awareness and control over sensitive information. At the same time, black-box integration testing confirms that several safeguards function as intended under default configurations: common personal identifiers are not promoted into user-facing metadata, and deletion requests are consistently propagated across both primary storage and search indices in accordance with GDPR Article 17.

Overall, Docspell can support privacy-preserving document management when carefully configured and responsibly operated. However, its privacy guarantees remain highly dependent on deployment and governance choices, particularly due to reliance on external OCR components that introduce trust-boundary considerations beyond software design alone. To

better align with Privacy-by-Design principles, it is recommended that Docspell should integrate privacy regression tests into continuous testing, provide a transparent interface for managing external processing, and adopt stricter default data-minimization strategies. These measures would strengthen privacy assurances while preserving core system functionality.

Disclaimer

This report was edited with the assistance of OpenAI's ChatGPT (version GPT-5.1), which was used to condense pre-written content, refine wording, and provide limited support with coding-related tasks. All substantive content, analysis, and conclusions remain the work of the authors, with AI serving only as an editorial and technical aid.

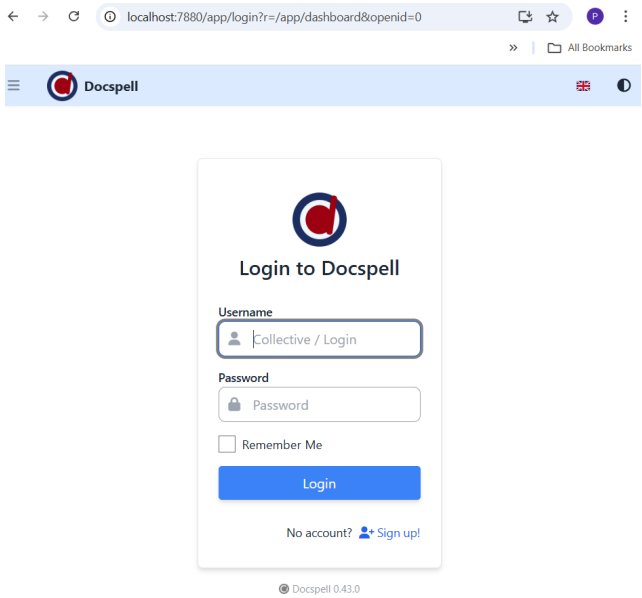
References

1. Docspell Contributors. (2025). Docspell: Personal document management system. https://docspell.org/
2. Docspell Contributors. (2025). Docspell official documentation. https://docspell.org/docs/
3. Deng, M., Wuyts, K., Scandariato, R., Preneel, B., & Joosen, W. (2011). A privacy threat analysis framework: Supporting the elicitation and fulfillment of privacy requirements.
4. Requirements Engineering, 16(1), 3–32. https://doi.org/10.1007/s00766-010-0115-2
5. Krekel, H., & pytest-dev team. (2025). Pytest: Helps you write better programs.https://docs.pytest.org/
6. Kettner, E., & Contributors. (2025). Docspell source code repository. https://github.com/eikek/docspell
7. LINDDUN Privacy Framework. (2025). LINDDUN threat trees. https://linddun.org/threat-trees/
8. European Union. (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). https://eur-lex.europa.eu/eli/reg/2016/679/oj
9. LINDDUN Privacy Framework. (2025). LINDDUN: A privacy threat modeling methodology. https://www.linddun.org/
10. Google. (2025). Tesseract OCR. https://github.com/tesseract-ocr/tesseract

Appendix

Running from source

```
[+] Running 5/5
 ✓ Container postgres_db          Running
 ✓ Container docspell-restserver  Running
 ✓ Container docspell-joex        Running
 ✓ Container docspell-consumedir  Running
 ✓ Container docspell-solr        Running
```



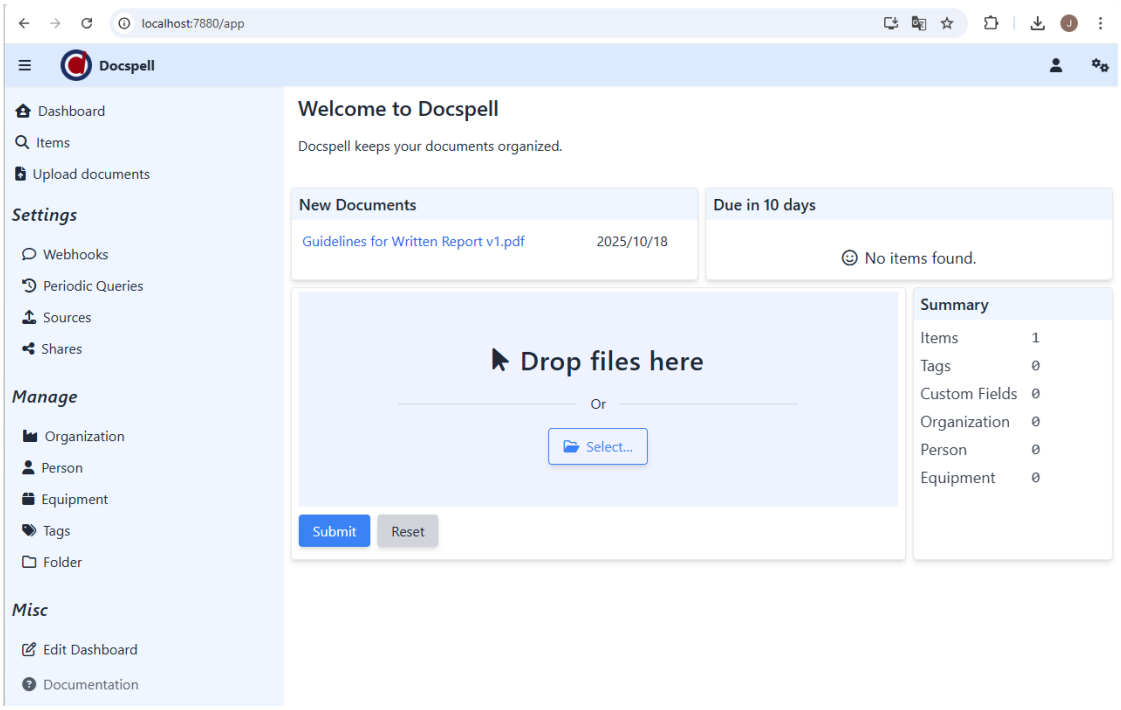


Figure 11,12,13 : deployment test

A Technical Verification: Threat (NLP Metadata Leakage)

A.1 Test Environment Setup

The verification was performed on a local instance of Docspell (v0.43.0). The setup isolates the system under test to ensure the REST API and the processing backend (Joex) function correctly within a controlled environment.

- **System Under Test (SUT):** Docspell Java application executed via terminal.
- **Test Runner:** A Python environment using pytest, acting as an external REST API client.
- **Data Set:** A deterministic "Sensitivity Bank" containing generated fake PII.

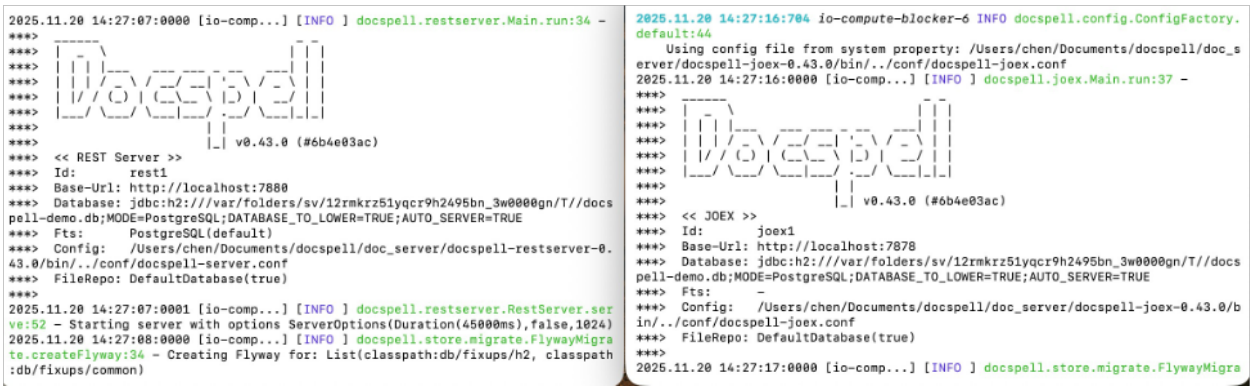


Figure 14: a test runner interacting with a Docspell REST API and Joex backend

A.2 Implementation Rationale

The goal is to detect if the Natural Language Processing (NLP) component incorrectly identifies sensitive strings as entities and promotes them to global metadata tags.

A.2.1 Deterministic Data Generation

To maintain test consistency, the script avoids random entropy and uses a fixed seed to generate PII patterns. This ensures that every test run uses the exact same sensitive strings for comparison.

```
def build_pseudo_sensitive_dataset(size=10):
    rng = random.Random(4242)  # Fixed seed for repeatability
    # ... logic to generate fake SSNs and IBANs ...
    leak_terms = {
        ssn.lower(),
        ssn.replace("-", ""),
        iban.lower(),
        email.lower()
    }
    return dataset
```

Listing 1: Snippet from test_privacy_threats.py: Deterministic Dataset

A.2.2 The Arrange-Act-Assert Pattern

The verification logic is divided into three distinct phases to ensure a clean state before and after the data injection.

```
# Identify only the tags added during the current test execution
new_tags = tags_after_set - tags_before_set

# Check if any new tag matches a sensitive 'leak term'
leaked_tags = [val for val in sensitive_values_to_check if val in new_tags]

if leaked_tags:
    pytest.fail(f"Privacy leak confirmed. Sensitive data was added as tags: {
    leaked_tags}")
```

Listing 2: Snippet: Leak Detection and Assertion

A.3 Sensitivity Bank Specification

The "Sensitivity Bank" provides standardized patterns to test different extraction capabilities of the NLP pipeline:

Data Type	Pattern Example	Rational for Inclusion
Financial	DE02000...00	Tests IBAN extraction/tagging risks.
Personal	case101@gmail.com	Validates contact information filtering.

A.4 Verification Results

The test suite executed against 10 unique pseudo-documents. The result was **PASSED**, indicating that Docspell's default NLP configuration successfully ignored these high-sensitivity patterns.

B Technical Verification: Threat (Document Upload and Job Queue Privacy)

B.1 Test Environment Setup

Same as the previous one.

B.2 Implementation Rationale

The purpose of this verification is to ensure that Docspell prevents privacy leakage by separating user identity from document metadata and internal job processing. The test specifically evaluates whether uploader information or identifier reuse is exposed through the API or job queue.

B.2.1 The Arrange-Act-Assert Pattern

- **Arrange:** A valid authentication token is obtained, and two test documents are prepared.
- **Act:** The documents are uploaded via POST /sec/upload/item. Document identifiers are retrieved through the search API, and the job queue state is queried via /sec/queue/state.
- **Assert:** The test verifies that document metadata contains no uploader information, that document IDs are unique, and that document IDs do not match job IDs. Job queue entries are confirmed to expose only operational details without user-identifying metadata.

```
def test_id():
    token = login_and_get_token(USERNAME, PASSWORD)
    document_ids = []

    print("=== Uploading Documents ===")
    for doc in UPLOAD_TEST_DATASET:
        file_path = DATA_DIR / f"{doc['label']}.txt"
        file_path.write_text(doc["content"], encoding="utf-8")
        upload_resp = upload_file(file_path, token)
        assert upload_resp.get("success") is True

        doc_id = get_document_id_by_name(file_path.name, token)
        document_ids.append(doc_id)

        metadata = get_document_metadata(doc_id, token)
        uploader = metadata.get("createdBy") or metadata.get("uploader")
        print(f"Document '{file_path.name}' -> ID: {doc_id}, Uploader:
{uploader}")

        # Cleanup local test file
        file_path.unlink(missing_ok=True)
```

```
    print("\n=== Checking Job Queue ===")
    jobs = get_job_queue(token)
    for state, job_list in jobs.items():
        print(f"State: {state} — {len(job_list)} jobs")
        for job in job_list:
            job_id = job.get("id")
            item_id = job.get("itemId")
            item_name = job.get("name")
            print(f"  Job ID: {job_id}, Item ID: {item_id}, Item name:
{item_name}")
            if item_id:
                # Ensure document ID is not equal to job ID
                assert item_id not in document_ids or item_id != job_id
```

Listing 3: Snippet from test_id.py

B.3 Distinct Identifiers

Artifact Type	Name	Identifier
Document	doc1.txt	VYuUbWciXRo-NLgw5iNdKyt-j74RPCjrZAt-HeHbgN6DiV
Job	doc1.txt	2cQku61xwg7-N4WKhqvznLS-3ajaez7FJP1-hjK6sicJkE2
Document	doc2.txt	Dz2NPsBHYd1-Uo8BiPxMUVS-4WvhGsztAo7-d6SqTYxnjSB
Job	doc2.txt	9sSxqXabZGX-wv4YrxJV8Vk-BqbtZ55584q-5hFU1WJNXDH

B.4 Verification Results

The test executed successfully. Both documents were processed correctly, no uploader information was exposed, and document identifiers remained distinct from job identifiers.